Design Document

Project Description:

Garage Tracker is a full-stack web application for tracking vehicle maintenance, built for the CS5610 Web Development class at Northeastern University. It is designed for anyone who owns one or more vehicles and wants their maintenance history in one place instead of a glovebox full of receipts. Most car owners keep service records in scattered receipts and oil-change stickers, so it's hard to remember what was done, what it cost, or what's due next.

There are two main pages:

1. **Vehicles ("Your Garage"):** Where users manage the cars they own. You can add a vehicle with its make, model, year, nickname, current mileage, purchase price, and status (Active / In Repair / Garaged / Sold), then browse, search, and filter the garage by status, make, or year. Opening a vehicle shows a detailed view with all its information, a quick control to update the mileage, and the option to edit or delete it.

2. **Services (Maintenance History & Costs):** Where users log each service performed on a vehicle (date, service type, mileage, cost, shop, and a 1–5 shop rating) and review it. The page filters service history by vehicle, type, and date range, and shows useful summaries: total maintenance spend per vehicle, cost-per-mile, monthly spending, and which services are coming due based on mileage.

The app is built with HTML5, CSS3, and vanilla JavaScript (ES6) on the frontend, with Node.js and Express on the backend and MongoDB (native driver) for storage. It uses two MongoDB collections — Vehicles and Services — each supporting full CRUD. The two are linked: each service stores the _id of the vehicle it belongs to, so the maintenance history always points back to a real car in the garage.

The goal of this project was to give car owners a single, organized record of what they own and what it has cost to maintain, while demonstrating a full-stack web application with a real database, a REST API, and a client-side-rendered frontend.

User Personas/Stories:

Persona 1: The Multi-Car Owner

Name: Marcus Bell, 38 **Role:** Homeowner with a three-car driveway **Location:** Austin, TX

Background: Marcus owns a daily-driver sedan, a weekend project car he's slowly restoring, and his partner's SUV. He's handy and does some of the work himself, but he has never had a single place to keep track of all three cars. He constantly loses track of which car needs what, what condition each one is in, and what he paid for it. He wants a clear, organized view of his whole garage so maintenance stops being a pile of guesses.

Scenario: Marcus is thinking about selling the project car. He opens Garage, filters his garage down to that one vehicle, and opens its detail view to check the record at a glance. He updates the odometer to today's reading using the quick mileage control, then changes the car's status so his list reflects that it's on its way out. Later he adds a car he just bought so the garage stays complete.

User Stories:

1. As Marcus, I want to add a vehicle with its make, model, year, nickname, current mileage, purchase price, and status, so I can keep an accurate record of everything in my garage.
2. As Marcus, I want to browse and filter my vehicles by status, make, and year, so I can quickly pull up the car I'm looking for.
3. As Marcus, I want to open a vehicle's detail view to see all its information in one place, so I can check a car's record at a glance.
4. As Marcus, I want to edit a vehicle's details and update its current mileage as it changes, so my records stay accurate.
5. As Marcus, I want to delete a vehicle I've sold or no longer own, so my garage stays current.

Persona 2: The Diligent Logger

Name: Priya Nair, 31 **Role:** Full-time professional who maintains her own car **Location:** Boston, MA

Background: Priya works full time and is meticulous about car upkeep. She keeps every receipt and logs each oil change, brake job, and tire rotation the day it happens. What frustrates her is not having a way to see her spending add up over time, so maintenance costs always feel like a surprise. She wants to filter her service history and watch where the money is actually going so she can budget for upkeep instead of being caught off guard.

Scenario: After an expensive month of repairs, Priya logs the latest service with its date, type, mileage, cost, shop, and a rating for the shop. She then filters her history to that vehicle to review the recent work, and pulls up the monthly spend summary to see which months were the costly ones and plan ahead.

User Stories:

1. As Priya, I want to log a service with the date, service type, mileage, cost, shop, and a shop rating, so I can build a complete maintenance history.
2. As Priya, I want to view services filtered by vehicle, service type, and date range, so I can review the work that's been done recently.
3. As Priya, I want to see a monthly summary of maintenance spend, so I understand where my upkeep money is going.

Persona 3: The Future Seller

Name: Sofia Reyes, 29 **Role:** Car owner planning to sell next year **Location:** Jersey City, NJ

Background: Sofia plans to sell her car next year and knows from experience that a documented service history raises the asking price and reassures buyers. She isn't especially technical about cars, but she's organized and wants proof that the car was well cared for. She's looking for a clean, complete maintenance record and a clear picture of what the car has actually cost her, plus confidence that no scheduled maintenance ever slipped.

Scenario: As the sale gets closer, Sofia opens the cost summary to see her car's total maintenance spend and its cost-per-mile, so she can price it confidently. She skims the due-soon view to make sure nothing is overdue before a buyer inspects it, and cleans up a service record she'd entered with the wrong cost so the history is accurate and presentable.

User Stories:

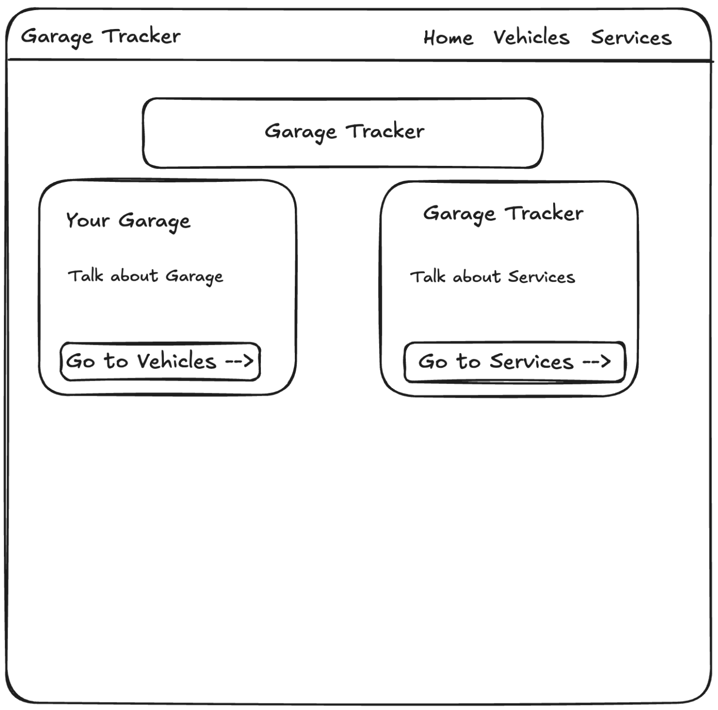
1. As Sofia, I want to see a cost summary per vehicle showing total maintenance spend and cost-per-mile, so I know what my car really costs me.
2. As Sofia, I want to see which services are due soon for each vehicle based on mileage intervals, so scheduled maintenance never slips.
3. As Sofia, I want to edit or delete service records I logged incorrectly, so my history stays accurate and presentable to buyers.

Division of Work:

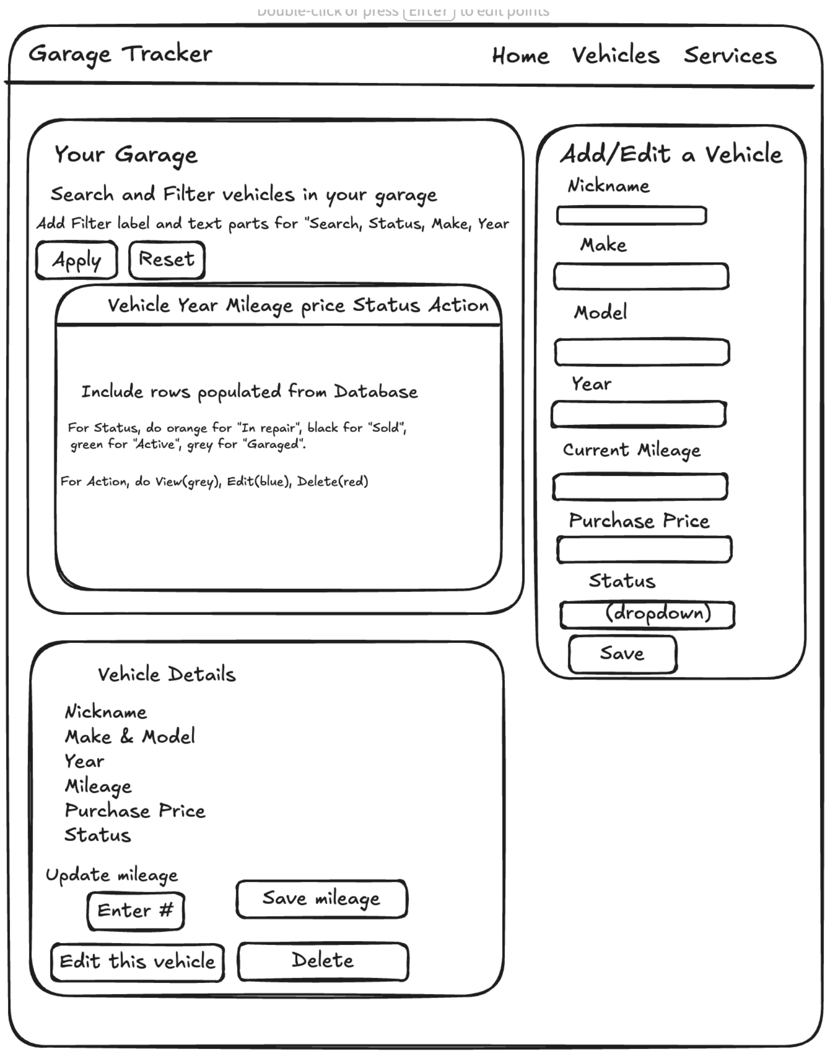
Nipun — Garage Management Feature (Full Stack): Garage page (browse, search, filter by status/make/year), vehicle detail view, add/edit vehicle form, mileage update control, and delete controls. All Express CRUD routes for the Vehicles collection, including filtering, and the seed data for vehicle records.

Khush — Maintenance History & Costs Feature (Full Stack): Service logging form with shop rating, service history page (filter by vehicle, type, and date range), edit/delete controls, cost summary views (monthly spend, total spend per vehicle, cost-per-mile), and due-soon maintenance view. All Express CRUD routes for the Services collection, including aggregation for spend summaries, cost-per-mile, and due-soon computation, and the seed data for service records.

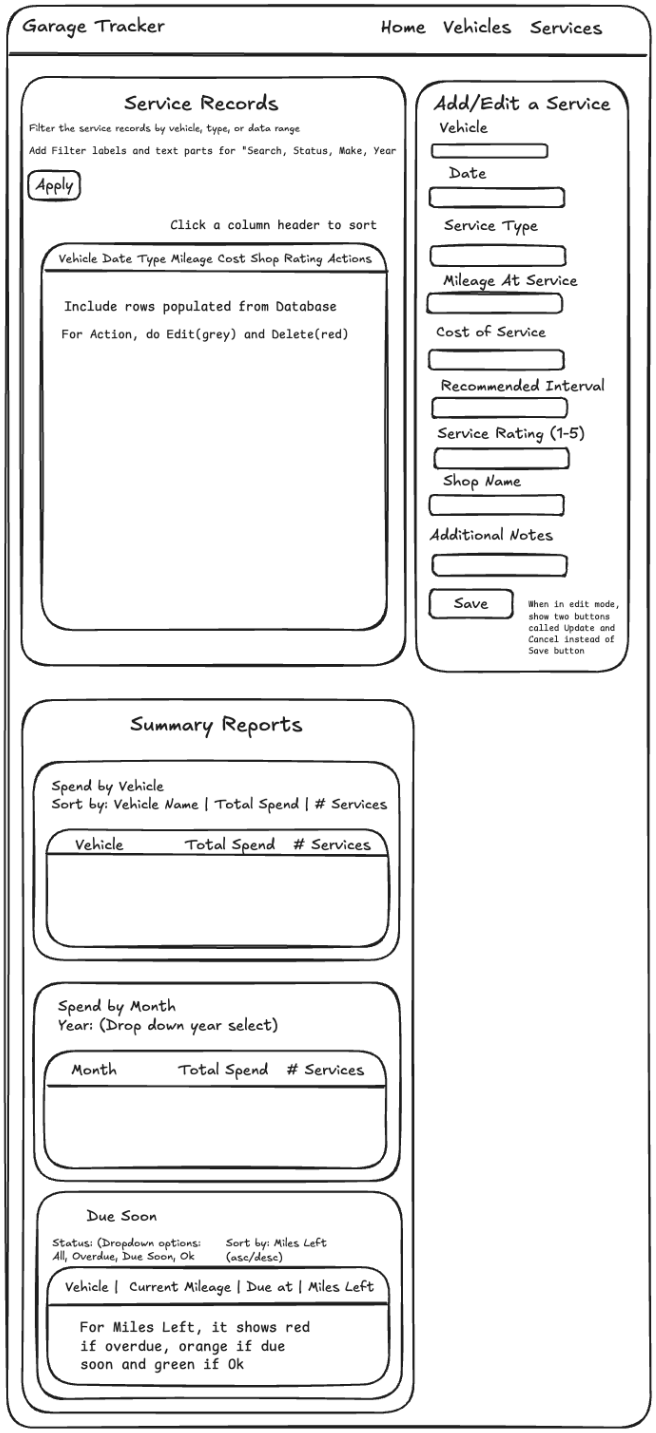
Design Mockups:



Home page (landing): a top nav bar (Garage Tracker brand on the left; Home, Vehicles, and Services links on the right). Below it, a centered page title, then two side-by-side cards introducing the app's two halves: “Your Garage” describing the vehicles feature with a "Go to Vehicles" button, and a "Service Records" card describing the maintenance feature with a "Go to Services" button. The cards give a first-time visitor a one-line sense of each side and a direct way in.



Vehicles page ("Your Garage"): a top nav bar (Garage Tracker brand, links to Services and Vehicles). Below it, two columns. The left column holds the filter controls (search box, status dropdown, make field, year field) above a scrollable table of vehicles (nickname + make/model, year, mileage, price, status badge, and View / Edit / Delete actions per row), with a Vehicle Details panel beneath it that fills in when a row is viewed (all fields plus a quick mileage-update control and a delete button). The right column is the add/edit form (one form for both adding and editing).



Services page (Maintenance History & Costs): the same nav bar and two-column layout. The left column holds the service history list with its filters (vehicle, type, date range) and the computed reports — total spend per vehicle, cost-per-mile, monthly spend, and the due-soon list (most urgent first). The right column is the add/edit form for logging a service (vehicle, date, type, mileage, cost, shop, rating, notes).

Technology Stack:

- **Frontend:** Vanilla JavaScript (ES6), HTML5, CSS3 (Bootstrap 5 for layout)
- **Backend:** Node.js + Express
- **Database:** MongoDB (Native Node.js Driver)
- **Data Requests:** Fetch API